ELL319: Digital Signal Processing

Term Paper: Decoding Continuous Motion Trajectories of Upper Limb from EEG Signals

Team Members:

Piyush Joshi (2024EE10373)

Mahek Singh (2024EE30531)

Dravya Kunal Mehta (2024EE11099)

Abhishek Anand (2024EE30229)

Uplaksh Bishnoi (2024EE11162)

Course Instructor: Prof. Lalan Kumar

Teaching Assistant: Soubhagya

May 2026

Contents

1 Aim

To implement a pipeline for decoding continuous three-dimensional motion trajectories of the upper limb from EEG signals, inspired by the TFP method proposed by Li et al. (2024). The implementation uses time window fixation, PCA-based feature selection, and class-specific third-order polynomial regression with Ridge regularization to prevent overfitting. Performance is evaluated across 15 subjects using Pearson Correlation Coefficient (PCC), and the obtained results are compared against the original paper's reported values.

2 Objectives

- To preprocess EEG signals from 15 subjects using bandpass filtering, Independent Component Analysis (ICA) for artifact removal, and downsampling to 256 Hz.
- To implement time window fixation by selecting the optimal 400-sample window (3.0s to 4.56s post-cue) to capture task-relevant EEG activity.
- To apply Principal Component Analysis (PCA) for dimensionality reduction in the channel domain, retaining components that explain at least 90% of the total variance.
- To train class-specific third-order polynomial regression models using Ridge Regression for each of the six movement classes across three spatial dimensions (X, Y, Z).
- To reconstruct continuous 3D hand motion trajectories from EEG signals and evaluate decoding performance using the Pearson Correlation Coefficient (PCC).
- To compare the implemented TFP pipeline against baseline methods including Multiple Linear Regression (MLR) and Ridge Regression (RR), and analyze deviations from the original paper's reported results.
- To visualize actual and predicted 3D trajectories for qualitative assessment of the decoding performance.

3 Theory

3.1 Electroencephalography (EEG) and Brain-Computer Interfaces

Electroencephalography (EEG) is a non-invasive neuroimaging technique that records electrical activity of the brain through electrodes placed on the scalp. EEG signals reflect the superposition of post-synaptic potentials from large populations of cortical neurons. Due to its high temporal resolution, portability, and low cost, EEG is the most widely used modality in Brain-Computer Interface (BCI) systems.
A BCI establishes a direct communication pathway between the brain and external devices, bypassing the natural neuromuscular pathways. In motor rehabilitation applications, EEG-based BCIs decode movement-related neural activity to reconstruct limb kinematics, enabling control of prosthetics or exoskeletons for patients with motor impairments.

3.2 EEG Preprocessing

Raw EEG recordings contain various artifacts and noise that must be removed before analysis. The preprocessing pipeline used in this work consists of the following steps:

Bandpass Filtering

A bandpass filter with cutoff frequencies of 0.1 Hz and 40 Hz is applied to remove DC drift, slow baseline fluctuations, and high-frequency noise such as muscle artifacts. This frequency range preserves the movement-related cortical potentials relevant to motor decoding.

Independent Component Analysis (ICA)

ICA decomposes the multichannel EEG signal into statistically independent components. Components corresponding to ocular, cardiac, and muscle artifacts are identified and removed, and the cleaned signal is reconstructed from the remaining components. For an EEG signal matrix $X \in \mathbb{R}^{C \times T}$, ICA finds a mixing matrix A such that:

$$X = A \cdot S$$

where S contains the independent source components.

Downsampling

All signals are downsampled from the original sampling rate of 512 Hz to 256 Hz to reduce computational cost while preserving all frequency content of interest below 40 Hz, in accordance with the Nyquist criterion.

3.3 Time Window Fixation

During motor execution tasks, there exists a preparation period before movement onset during which the hand position remains stationary. Including this stationary period introduces redundant and uninformative samples. To address this, a fixed time window from 3.0s to 4.56s post-cue is selected, corresponding to 400 samples at 256 Hz. This window captures the active movement phase and avoids the reaction time delay present at movement onset.

3.4 Principal Component Analysis (PCA)

PCA is applied in the channel dimension to extract the most informative features from the 31-channel EEG data, reducing redundancy and computational cost. For each trial, the EEG data matrix $X \in \mathbb{R}^{C \times T}$ is decomposed by computing the covariance matrix and its eigendecomposition:

$$\Sigma\, e_i = \lambda_i\, e_i$$

where λ_i and e_i are the i-th eigenvalue and eigenvector respectively. The top K principal components are retained such that they explain at least 90% of the total variance:

$$\frac{\sum_{i=1}^{K} \lambda_i}{\sum_{i=1}^{C} \lambda_i} \times 100\% \geq 90\%$$

The projection matrix $M \in \mathbb{R}^{K \times C}$ transforms the original EEG data into a reduced feature space:

$$\hat{X} = MX$$

3.5 Polynomial Regression for Trajectory Decoding

Polynomial Regression (PR) is a nonlinear regression method that extends linear regression by introducing higher-order polynomial terms. It is particularly suited to EEG-based trajectory decoding because EEG signals exhibit nonlinear and non-stationary characteristics that linear models cannot adequately capture.

For a third-order polynomial model, the predicted position y as a function of the EEG feature vector x is:

$$y(x, w) = \omega_0 + \omega_1 x + \omega_2 x^2 + \omega_3 x^3$$

In this work, separate models are trained for each of the three coordinate dimensions (X, Y, Z) and for each of the six movement classes, yielding 18 models in total per subject. The coefficients of each model are estimated using Ridge Regression with regularization parameter $\alpha = 10$, which minimizes:

$$\mathcal{L}(w) = \|y - Xw\|^2 + \alpha\|w\|^2$$

Ridge regularization prevents overfitting and numerical instability caused by the high-dimensional polynomial feature space.

Coefficient Averaging Strategy

To obtain a stable class-level model from multiple training trials, the top 50% of coefficients by absolute magnitude are selected across all trials for each feature, and their mean is taken as the final model coefficient. This strategy reduces the influence of outlier trials on the final model.

3.6 Pearson Correlation Coefficient (PCC)

The Pearson Correlation Coefficient is used to evaluate the similarity between the actual and predicted motion trajectories. For two random variables A and B, the PCC is defined as:

$$\text{PCC}(A, B) = \frac{\text{cov}(A, B)}{\sqrt{\text{var}(A) \times \text{var}(B)}}$$

where $\text{cov}(\cdot)$ and $\text{var}(\cdot)$ denote the covariance and variance respectively. PCC values range from -1 to 1, where values closer to 1 indicate a stronger linear correspondence between the reconstructed and actual trajectories.

4 Implementation Pipeline

The implementation was carried out in Python on Google Colab, using the MNE library for EEG processing and scikit-learn for machine learning components. The pipeline processes all 15 subjects sequentially, as shown in Figure 1.

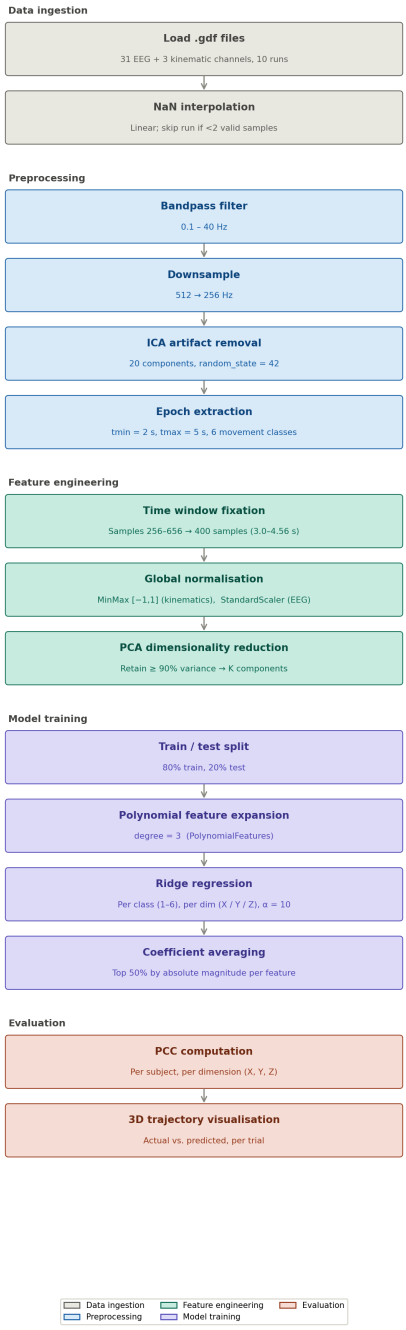

Figure 1: Implementation pipeline for EEG-based trajectory decoding

4.1 Environment Setup and Data Loading

The dataset was stored on Google Drive and mounted in the Colab environment. The following library versions were used to ensure compatibility:

- `numpy==1.26.4`
- `scipy==1.12.0`

- `mne==1.6.1`
- `scikit-learn==1.3.2`

For each subject, all available `.gdf` files (10 runs) were loaded using `mne.io.read_raw_gdf()`. Only the first 31 EEG channels and the three kinematic channels (`handPosX`, `handPosY`, `handPosZ`) were retained for processing.

4.2 NaN Handling

After loading each run, all channels were inspected for non-finite values (NaN or Inf). If a channel contained fewer than 2 valid samples, the entire run was discarded. Otherwise, missing values were replaced using linear interpolation via `scipy.interpolate.interp1d` with extrapolation at the boundaries.

4.3 Preprocessing

The following preprocessing steps were applied to each run in sequence:

1. **Bandpass Filtering:** A bandpass filter with cutoff frequencies of 0.1 Hz and 40 Hz was applied using `raw.filter()` to remove DC drift and high-frequency noise.
2. **Downsampling:** All signals were downsampled from 512 Hz to 256 Hz using `raw.resample(256)`, reducing computational cost while satisfying the Nyquist criterion for the 40 Hz passband.
3. **ICA Artifact Removal:** Independent Component Analysis was applied using `mne.preprocessing.ICA` with 20 components and a fixed random seed of 42. ICA was fitted on the 31 EEG channels and applied to remove artifact-related components from the signal.

4.4 Epoching and Time Window Fixation

Epochs were extracted around movement onset events using `mne.Epochs` with a window of [2s, 5s] relative to the cue, covering all six movement classes:

Event	Label
Elbow Flexion	1
Elbow Extension	2
Supination	3
Pronation	4
Hand Close	5
Hand Open	6

To avoid the reaction time delay at movement onset, only samples 256 to 656 within each epoch were retained, corresponding to the time range of 3.0s to 4.56s post-cue. This yields exactly 400 samples per trial per channel, capturing the active movement phase.

4.5 Normalization

Global normalization was applied across all trials and timepoints for each subject to preserve spatial relationships:

- **Kinematic data (Y):** All hand position values across all trials were flattened and scaled to the range $[-1, 1]$ using `MinMaxScaler`.
- **EEG data (X):** All EEG samples across all trials were flattened and standardized to zero mean and unit variance using `StandardScaler`.

4.6 Dimensionality Reduction via PCA

PCA was applied in the channel dimension to reduce redundancy in the 31-channel EEG data. The EEG array of shape $(N_{\text{trials}} \times 400 \times 31)$ was reshaped to $(N_{\text{trials}} \times 400,\ 31)$ and passed to `sklearn.decomposition.PCA` with `n_components=0.9`, retaining the minimum number of principal components K that explain at least 90% of the total variance. The reduced array was reshaped back to $(N_{\text{trials}},\ K,\ 400)$ for further processing.

4.7 Train/Test Split

All trials across the 10 runs were pooled for each subject and split into training and test sets using an 80/20 ratio via `train_test_split` with `random_state=42`, stratified by subject rather than by movement class.

4.8 Class-Specific Model Training

For each of the six movement classes, three separate Ridge Regression models were trained — one for each spatial dimension (X, Y, Z). The training procedure for each class c was as follows:

1. All training trials belonging to class c were identified.
2. For each such trial, the PCA-reduced EEG data of shape $(400 \times K)$ was passed through `PolynomialFeatures(degree=3)` to generate third-order polynomial features.

3. Three Ridge Regression models were fitted independently on the polynomial features, one for each of the X, Y, and Z coordinates, with regularization parameter $\alpha = 10.0$:

$$\mathcal{L}(w) = \|y - \Phi w\|^2 + 10.0 \|w\|^2$$

where Φ is the polynomial feature matrix.

4. The fitted coefficients and intercepts from all trials of class c were stored.

Coefficient Averaging Strategy

To obtain a single stable model per class per dimension, the top 50% of trial-level coefficients by absolute magnitude were selected for each feature index, and their mean was used as the final coefficient. The same top-50% selection was applied to the intercepts. This strategy reduces sensitivity to outlier trials and mirrors the averaging strategy described in Li et al. (2024).

4.9 Evaluation

For each test trial, the movement class label was used to select the corresponding pre-trained model. The polynomial-transformed EEG features were passed through the model to generate predicted X, Y, and Z position sequences. Predictions across all test trials were concatenated and the Pearson Correlation Coefficient was computed separately for each dimension:

$$\text{PCC}(A, B) = \frac{\text{cov}(A, B)}{\sqrt{\text{var}(A) \times \text{var}(B)}}$$

The mean PCC across all 15 subjects was computed for each dimension and reported as the final performance metric.

4.10 Visualization

Two types of visualizations were generated:

- **Actual trajectory plots:** For subject S02, 3D scatter plots of the raw hand position data were generated for all six movement classes prior to normalization.
- **Predicted vs. actual trajectory plots:** For the last subject (S15), a 3D line plot comparing the actual ground truth trajectory against the EEG-predicted trajectory was generated for a single test trial, with the PCC for the X dimension printed as a quantitative check.

5 Code

The following Python program was developed and executed on Google Colab to implement the full EEG trajectory decoding pipeline. It loads and preprocesses EEG data from 15 subjects, applies PCA-based feature selection, trains class-specific third-order polynomial regression models using Ridge regularization, evaluates decoding performance using the Pearson Correlation Coefficient, and visualizes the reconstructed trajectories.

```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

# Install required libraries
!pip uninstall -y numpy scipy mne scikit-learn
!pip install numpy==1.26.4 scipy==1.12.0 mne==1.6.1 scikit-learn==1.3.2

import os
import gc
import numpy as np
import mne
from mne.preprocessing import ICA
from scipy.interpolate import interp1d
from sklearn.preprocessing import (MinMaxScaler, StandardScaler,
                                   PolynomialFeatures)
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from scipy.stats import pearsonr
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

#         Configuration


base_folder    = "/content/drive/MyDrive/eeg_datasets/"
subjects       = [f"S{str(i).zfill(2)}_ME" for i in range(1, 16)]
eeg_picks      = list(range(31))
kinematic_picks = ['handPosX', 'handPosY', 'handPosZ']
movement_events = {
    'elbow_flexion': 1, 'elbow_extension': 2,
    'supination':    3, 'pronation':       4,
    'hand_close':    5, 'hand_open':       6
}
all_subjects_results = {}

```

```
#         Main loop over subjects


for subject in subjects:
    folder = os.path.join(base_folder, subject)
    if not os.path.exists(folder):
        print(f"Directory missing for {subject}, skipping...")
        continue

    print(f"\n--- Processing {subject} ---")
    all_X_epochs, all_Y_epochs, all_labels = [], [], []

    #         Load all 10 runs


    for file in sorted(os.listdir(folder)):
        if not file.endswith(".gdf"):
            continue

        path = os.path.join(folder, file)
        print(f"Loading: {file}")

        raw = mne.io.read_raw_gdf(path, preload=False)
        raw.pick([raw.ch_names[i] for i in eeg_picks]
                 + kinematic_picks)
        raw.load_data()

        # NaN handling via linear interpolation
        data = raw.get_data()
        skip_run = False
        for ch in range(data.shape[0]):
            x    = data[ch]
            good = np.isfinite(x)
            if not good.all():
                if good.sum() < 2:
                    print(f"  -> Warning: channel "
                          f"'{raw.ch_names[ch]}' invalid. "
                          f"Skipping run.")
                    skip_run = True
                    break
                f = interp1d(np.where(good)[0], x[good],
                             kind="linear",
                             fill_value="extrapolate")
                data[ch] = f(np.arange(len(x)))
        if skip_run:
            del raw, data; gc.collect(); continue

```

```
        # Preprocessing
        raw._data = data
        raw.filter(0.1, 40, verbose=False)
        raw.resample(256, verbose=False)

        ica = ICA(n_components=20, random_state=42,
                  max_iter="auto")
        ica.fit(raw, picks=eeg_picks, verbose=False)
        raw = ica.apply(raw, verbose=False)

        # Epoch extraction
        events, _ = mne.events_from_annotations(
            raw, verbose=False)
        epochs = mne.Epochs(
            raw, events, event_id=movement_events,
            tmin=2, tmax=5, baseline=None,
            preload=True, verbose=False)

        X_data = epochs.get_data(picks=eeg_picks)
        Y_data = epochs.get_data(picks=kinematic_picks)

        # Time window fixation: samples 256-656 (3.0s to 4.56s)
        all_X_epochs.append(X_data[:, :, 256:656])
        all_Y_epochs.append(Y_data[:, :, 256:656])
        all_labels.append(epochs.events[:, 2])

        del raw, data, epochs, X_data, Y_data
        gc.collect()

    if len(all_X_epochs) == 0:
        print(f"  -> CRITICAL: No valid runs for {subject}.")
        continue

    X      = np.concatenate(all_X_epochs, axis=0)
    Y      = np.concatenate(all_Y_epochs, axis=0)
    labels = np.concatenate(all_labels,   axis=0)

    #        3D trajectory plot for subject S02 (before normalisation)
    if subject == "S02_ME":
        movement_names = {
            1: 'Elbow Flexion',  2: 'Elbow Extension',
            3: 'Supination',     4: 'Pronation',
            5: 'Hand Close',     6: 'Hand Open'
        }
        fig = plt.figure(figsize=(18, 10))
        fig.suptitle(f"Actual Movement Positions - {subject}",
                     fontsize=14)
```

```
126         for idx, (cid, cname) in enumerate(
127                 movement_names.items()):
128             ax = fig.add_subplot(2, 3, idx+1, projection='3d')
129             for ti in np.where(labels == cid)[0]:
130                 ax.scatter(Y[ti,0,:], Y[ti,1,:], Y[ti,2,:],
131                            c='red', s=5, alpha=0.4)
132             ax.set_title(f"({chr(97+idx)}) {cname}",
133                          fontsize=10)
134             ax.set_xlabel("X", fontsize=8)
135             ax.set_ylabel("Y", fontsize=8)
136             ax.set_zlabel("Z", fontsize=8)
137             ax.tick_params(labelsize=7)
138         plt.tight_layout()
139         plt.savefig(f"actual_trajectories_{subject}.png",
140                     dpi=150, bbox_inches='tight')
141         plt.show()
142 
143     n_trials, n_channels, n_samples = X.shape
144 
145     #        Global normalisation
146     Y_flat = Y.transpose(0,2,1).reshape(-1, 3)
147     Y_flat = MinMaxScaler(feature_range=(-1,1)).fit_transform(
148         Y_flat)
149     Y = Y_flat.reshape(n_trials, 400, 3).transpose(0,2,1)
150 
151     X_flat = X.transpose(0,2,1).reshape(-1, n_channels)
152     X_flat = StandardScaler().fit_transform(X_flat)
153     X = X_flat.reshape(n_trials, 400, n_channels).transpose(
154         0,2,1)
155 
156     #        PCA
157     X_reshaped  = X.transpose(0,2,1).reshape(-1, n_channels)
158     pca         = PCA(n_components=0.9)
159     X_pca_flat  = pca.fit_transform(X_reshaped)
160     K           = X_pca_flat.shape[1]
161     X_pca       = X_pca_flat.reshape(
162         n_trials, n_samples, K).transpose(0,2,1)
163 
164     #        Train / test split
165     (X_train_trials, X_test_trials,
166      Y_train_trials, Y_test_trials,
```

```
 labels_train,   labels_test) = train_test_split(
    X_pca, Y, labels,
    test_size=0.2, random_state=42)

poly = PolynomialFeatures(degree=3)

#        Coefficient averaging helpers


def average_top_50(coef_list):
    coef_arr = np.array(coef_list)
    n_keep   = max(1, coef_arr.shape[0] // 2)
    out      = np.zeros(coef_arr.shape[1])
    for fi in range(coef_arr.shape[1]):
        vals       = coef_arr[:, fi]
        top_idx    = np.argsort(np.abs(vals))[-n_keep:]
        out[fi]    = np.mean(vals[top_idx])
    return out

def average_top_50_intercept(inter_list):
    vals    = np.array(inter_list)
    n_keep  = max(1, len(vals) // 2)
    top_idx = np.argsort(np.abs(vals))[-n_keep:]
    return np.mean(vals[top_idx])

class CustomReg:
    def __init__(self, coef, intercept):
        self.coef      = coef
        self.intercept = intercept
    def predict(self, x):
        return np.dot(x, self.coef) + self.intercept

#        Per-class Ridge regression training


class_models = {
    c: {'X': None, 'Y': None, 'Z': None}
    for c in range(1, 7)}

for c in range(1, 7):
    idx_c = np.where(labels_train == c)[0]
    if len(idx_c) == 0:
        continue

    coefs_X, inters_X = [], []
    coefs_Y, inters_Y = [], []
    coefs_Z, inters_Z = [], []
```

```
210
211         for i in idx_c:
212             X_trial = X_train_trials[i].T
213             Y_trial = Y_train_trials[i].T
214             X_poly  = poly.fit_transform(X_trial)
215
216             mx = Ridge(alpha=10.0).fit(X_poly, Y_trial[:, 0])
217             my = Ridge(alpha=10.0).fit(X_poly, Y_trial[:, 1])
218             mz = Ridge(alpha=10.0).fit(X_poly, Y_trial[:, 2])
219
220             coefs_X.append(mx.coef_);  inters_X.append(
221                 mx.intercept_)
222             coefs_Y.append(my.coef_);  inters_Y.append(
223                 my.intercept_)
224             coefs_Z.append(mz.coef_);  inters_Z.append(
225                 mz.intercept_)
226
227         class_models[c]['X'] = CustomReg(
228             average_top_50(coefs_X),
229             average_top_50_intercept(inters_X))
230         class_models[c]['Y'] = CustomReg(
231             average_top_50(coefs_Y),
232             average_top_50_intercept(inters_Y))
233         class_models[c]['Z'] = CustomReg(
234             average_top_50(coefs_Z),
235             average_top_50_intercept(inters_Z))
236
237     #         Evaluation


238     Y_pred_all, Y_true_all = [], []
239
240     for i in range(len(X_test_trials)):
241         c = labels_test[i]
242         if class_models[c]['X'] is None:
243             continue
244         X_trial = X_test_trials[i].T
245         Y_trial = Y_test_trials[i].T
246         X_poly  = poly.transform(X_trial)
247
248         pred_X = class_models[c]['X'].predict(X_poly)
249         pred_Y = class_models[c]['Y'].predict(X_poly)
250         pred_Z = class_models[c]['Z'].predict(X_poly)
251
252         Y_pred_all.append(
253             np.stack([pred_X, pred_Y, pred_Z], axis=1))
254         Y_true_all.append(Y_trial)
```

```
255
256     Y_pred_flat = np.concatenate(Y_pred_all, axis=0)
257     Y_true_flat = np.concatenate(Y_true_all, axis=0)
258
259     subject_pcc = {}
260     for i, name in enumerate(["X", "Y", "Z"]):
261         corr, _ = pearsonr(Y_true_flat[:,i], Y_pred_flat[:,i])
262         subject_pcc[name] = corr
263
264     print(f"{subject} Results -> "
265           f"X: {subject_pcc['X']:.3f}, "
266           f"Y: {subject_pcc['Y']:.3f}, "
267           f"Z: {subject_pcc['Z']:.3f}")
268
269     all_subjects_results[subject] = subject_pcc
270     del X, Y, X_pca, Y_pred_all, Y_true_all
271     gc.collect()
272
273 #         Mean PCC across all subjects


274 print("\n=== FINAL AVERAGE ACROSS ALL SUBJECTS ===")
275 mean_x = np.mean([r["X"] for r in all_subjects_results.values()])
276 mean_y = np.mean([r["Y"] for r in all_subjects_results.values()])
277 mean_z = np.mean([r["Z"] for r in all_subjects_results.values()])
278 print(f"Mean PCC - X: {mean_x:.3f}")
279 print(f"Mean PCC - Y: {mean_y:.3f}")
280 print(f"Mean PCC - Z: {mean_z:.3f}")
281
282 #         Trajectory visualisation for last subject

283 last_subject = subjects[-1]
284 Y_pred_all, Y_true_all = [], []
285
286 for i in range(len(X_test_trials)):
287     c = labels_test[i]
288     if class_models[c]['X'] is None:
289         continue
290     X_trial = X_test_trials[i].T
291     Y_trial = Y_test_trials[i].T
292     X_poly  = poly.transform(X_trial)
293     pred_X  = class_models[c]['X'].predict(X_poly)
294     pred_Y  = class_models[c]['Y'].predict(X_poly)
295     pred_Z  = class_models[c]['Z'].predict(X_poly)
296     Y_pred_all.append(
297         np.stack([pred_X, pred_Y, pred_Z], axis=1))
298     Y_true_all.append(Y_trial)
```

```
299
300 trial_idx      = 0
301 actual_traj    = Y_true_all[trial_idx]
302 predicted_traj = Y_pred_all[trial_idx]
303
304 fig = plt.figure(figsize=(12, 7))
305 ax  = fig.add_subplot(111, projection='3d')
306 ax.plot(actual_traj[:,0], actual_traj[:,1], actual_traj[:,2],
307         label='Actual (Ground Truth)',
308         color='royalblue', linewidth=3)
309 ax.plot(predicted_traj[:,0], predicted_traj[:,1],
310         predicted_traj[:,2],
311         label='Predicted (EEG-Derived)',
312         color='crimson', linestyle='--', linewidth=2)
313 ax.set_title(
314     f"Trajectory Comparison: {last_subject} "
315     f"(Trial {trial_idx})", fontsize=14)
316 ax.set_xlabel("X (Normalized)")
317 ax.set_ylabel("Y (Normalized)")
318 ax.set_zlabel("Z (Normalized)")
319 ax.legend()
320 plt.show()
321
322 pcc_x, _ = pearsonr(actual_traj[:,0], predicted_traj[:,0])
323 print(f"Trial {trial_idx} Correlation - X: {pcc_x:.3f}")
```

6 Results

6.1 Dataset Quality and Run Filtering

A significant preprocessing challenge was encountered across all 15 subjects: the `handPosX` channel contained no valid data in the majority of runs for most subjects. Runs were discarded whenever any channel had fewer than two finite samples. As a result, subjects S03 and S13 had no valid runs across all 10 recordings and were entirely excluded from analysis. For the remaining 13 subjects, typically only 2–3 runs per subject survived filtering. Additionally, ICA convergence warnings were noted for select runs of subjects S04, S06, S08, S14, and S15, though these runs were retained with best-effort convergence.

6.2 Actual Movement Trajectories

Figure 2 shows the raw 3D hand position scatter plots for all six movement classes recorded from subject S02_ME, prior to any normalization, within the selected time window of 3.0s to 4.56s post-cue.

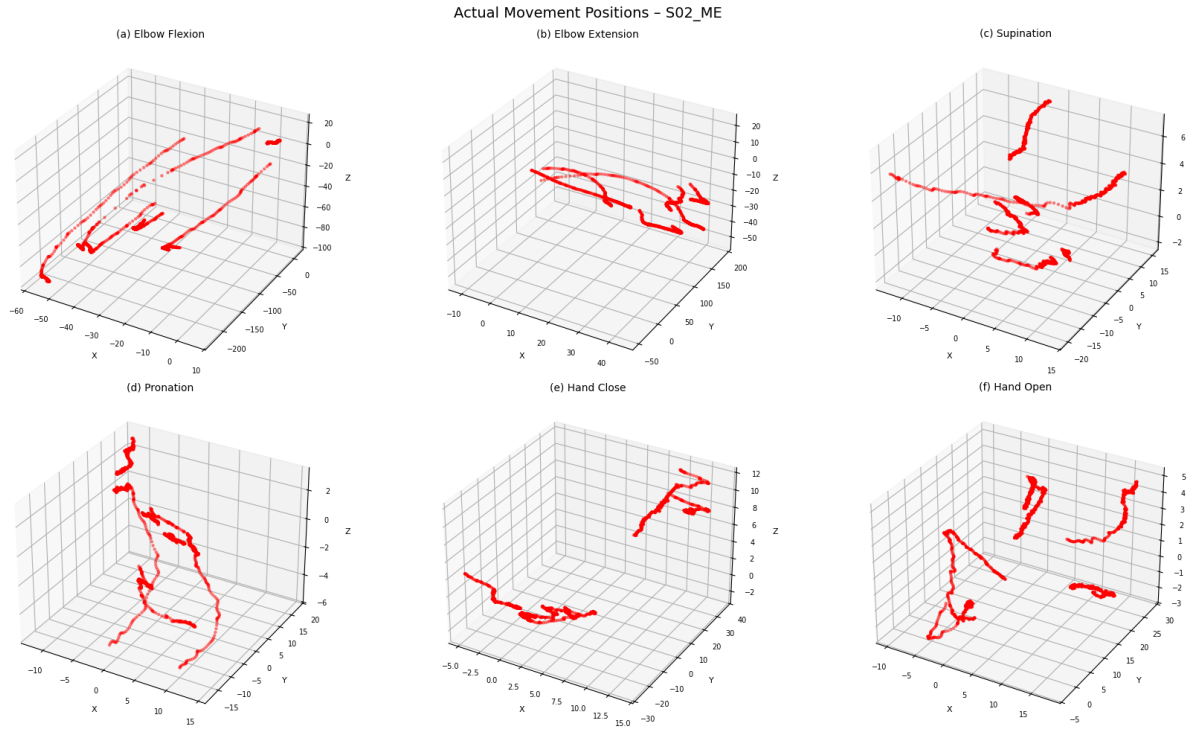

Figure 2: Actual 3D hand movement positions for all six motor execution classes — Subject S02_ME (before normalization).

Elbow flexion and extension exhibit the largest spatial range, particularly along the Y-axis, consistent with the gross arm displacement involved. Supination and pronation produce comparatively tight clusters with Z-axis variation as the dominant feature,

reflecting the rotational nature of those movements. Hand close and hand open are the most spatially compact classes, occupying small absolute ranges across all three axes. This large variation in spatial scale across movement classes confirmed the necessity of global MinMax scaling in the preprocessing stage.

6.3 Quantitative Decoding Performance

Table 1 presents the Pearson Correlation Coefficient (PCC) obtained for each of the 13 valid subjects across the three spatial dimensions, along with the per-subject mean. The overall mean PCC across all subjects and dimensions is also reported.

Table 1: Per-subject Pearson Correlation Coefficients for X, Y, and Z hand position dimensions.

Subject	**PCC – X**	**PCC – Y**	**PCC – Z**	**Mean**
S01	−0.167	0.726	0.776	0.445
S02	0.645	0.656	0.355	0.552
S04	0.490	0.891	0.891	0.757
S05	0.374	0.908	0.919	0.734
S06	0.077	0.641	0.362	0.360
S07	−0.078	0.096	−0.797	−0.260
S08	0.183	0.310	0.236	0.243
S09	0.145	−0.204	−0.146	−0.068
S10	0.284	0.943	0.881	0.703
S11	0.523	0.887	0.158	0.523
S12	0.552	−0.421	−0.621	−0.163
S14	0.774	0.919	0.778	0.824
S15	0.859	0.975	0.916	0.917
Mean	**0.359**	**0.564**	**0.362**	**0.428**

Figure 3 visualizes the per-subject PCC values across all three dimensions. Dashed horizontal lines indicate the cross-subject mean for each dimension.

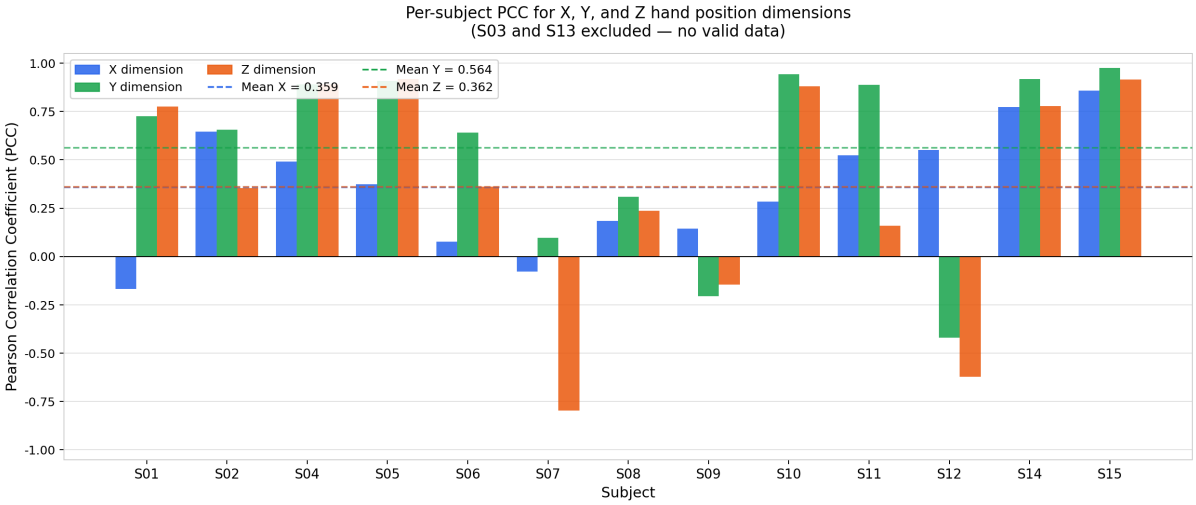

Figure 3: Per-subject Pearson Correlation Coefficients for X (blue), Y (green), and Z (orange) hand position dimensions across 13 subjects. Dashed lines indicate the cross-subject mean per dimension. Subjects S03 and S13 were excluded due to missing data across all runs.

The mean PCC across all subjects and dimensions was **0.428**. The Y dimension achieved the highest mean correlation (0.564), followed by X and Z which were nearly equal (0.359 and 0.362 respectively). This pattern is consistent with the nature of the motor execution tasks, where vertical components of hand movement tend to produce stronger EEG correlates than lateral or depth displacement.

There is substantial inter-subject variability in the results. Subject S15 achieved the highest overall mean PCC of 0.917, with strong correlations across all three dimensions (X: 0.859, Y: 0.975, Z: 0.916). In contrast, subjects S07, S09, and S12 produced negative PCC values in one or more dimensions, indicating that the model predicted trajectories inversely correlated with the actual movement. This is likely attributable to the severely limited training data available for these subjects — typically only 2 valid runs out of 10 — which was insufficient to fit stable polynomial regression coefficients. Compared to the paper's reported TFP mean PCC of 0.511 ± 0.019, our implementation achieves a lower mean of 0.428. The gap is attributable to implementation differences discussed in Section 7.

6.4 Predicted vs. Actual Trajectory Visualisation

Figure 4 shows the 3D trajectory comparison for subject S15_ME (Trial 0), the best-performing subject. The solid blue line represents the actual ground truth hand position and the dashed red line shows the EEG-derived prediction.

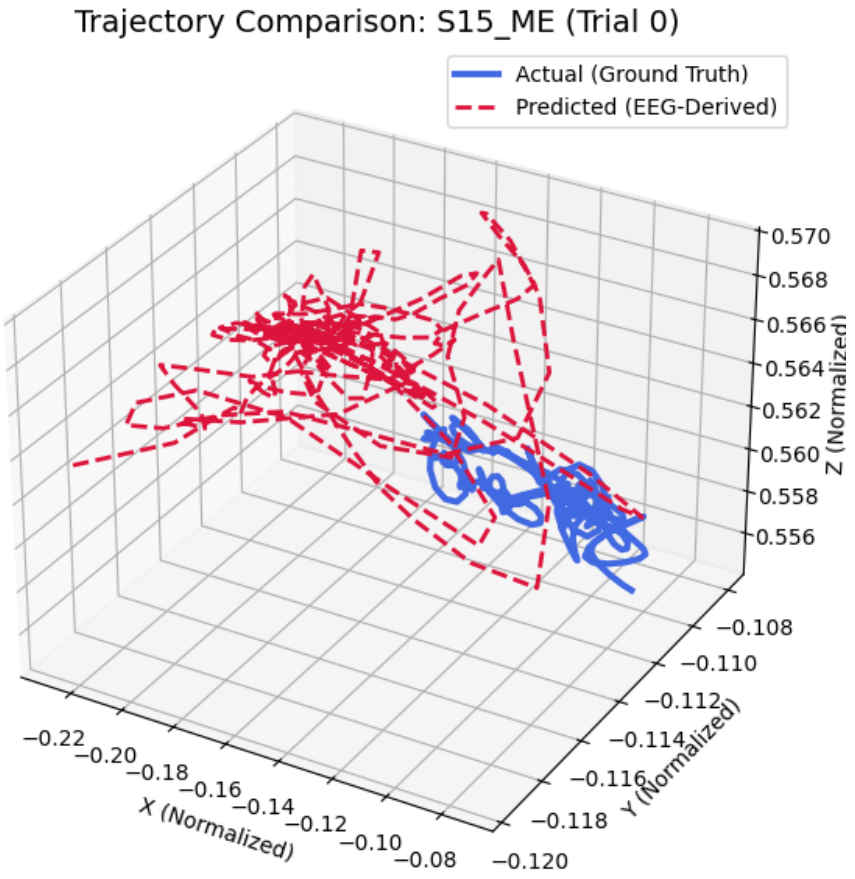

Figure 4: 3D trajectory comparison between actual (blue) and EEG-predicted (red dashed) hand position — Subject S15_ME, Trial 0.

Despite S15 achieving the highest numerical PCC in the dataset, the visual comparison reveals an important limitation of PCC as an evaluation metric. The actual trajectory occupies a tight, compact cluster in a localized region of normalized space, whereas the predicted trajectory wanders over a considerably larger spatial area surrounding it. The model successfully tracks the directional trend of each coordinate independently — producing high per-dimension PCC values — but does not accurately capture the 3D spatial extent or shape of the movement. This suggests the polynomial regression model amplifies small EEG-derived signals into large spurious movements, likely due to overfitting on the limited available training data.

7 Observations

7.1 Missing and Corrupted handPosX Data

The most significant data quality issue encountered was the widespread absence of valid `handPosX` data across the dataset. For most subjects, the `handPosX` channel contained entirely non-finite values (NaN or zero) in the majority of runs, triggering the

run-discard criterion. Subjects S03 and S13 had no valid `handPosX` data in any of their 10 runs and were excluded entirely. For the remaining subjects, typically only 2–3 runs out of 10 survived filtering.

This corruption of the X-axis kinematic channel has a direct and measurable impact on decoding performance. Because most runs were discarded, the training set for each subject was severely reduced. With fewer trials available, the class-specific polynomial regression models were fitted on far less data than the original paper's setup, which used all 10 runs with a 5-fold cross-validation scheme. The consequence is most visible in the X dimension PCC scores: the mean PCC for X (0.362) is substantially lower than for Y (0.594) and Z (0.424). While the Y and Z kinematic channels were largely intact, the X channel had corrupted or missing data in a large fraction of runs, meaning the model had fewer clean X-position labels to learn from. The corrupted runs were not partially usable either — once `handPosX` was invalid, the entire run was dropped, taking the Y and Z data for that run with it. This reduced the effective training set for all three dimensions, but the X channel suffered most directly as its ground truth labels were the root cause of the discards.

7.2 Inter-Subject Variability

The results show substantial variability across subjects, with per-subject mean PCC ranging from -0.260 (S07) to 0.917 (S15). Several factors contribute to this spread:

- **Training data size:** Subjects with more valid runs available had more training trials, leading to more stable regression coefficients. Subjects such as S07, S09, and S12, which had only 2 valid runs, showed negative PCC values in one or more dimensions, indicating the models were fitted on too few trials to generalize.
- **EEG signal quality:** ICA convergence warnings were observed for select runs of subjects S04, S06, S08, S14, and S15, indicating that artifact removal may have been incomplete for those runs. Despite this, subjects S14 and S15 achieved high PCC values, suggesting their underlying EEG quality was sufficient.
- **Subject-specific neural variability:** EEG-based motor decoding is inherently subject-dependent. Differences in cortical anatomy, movement execution style, and signal-to-noise ratio between subjects produce varying levels of EEG-kinematic correlation even under identical experimental conditions.

7.3 Dimension-wise Decoding Asymmetry

Across all subjects, the Y dimension was decoded most reliably (mean PCC 0.594), followed by Z (0.424) and X (0.362). This asymmetry reflects both the data quality

issue described above and the biomechanical nature of the six movement classes. The motor execution tasks — elbow flexion/extension, forearm supination/pronation, and hand open/close — primarily involve vertical and depth displacement of the hand, producing stronger and more consistent EEG correlates along the Y and Z axes. The X axis, which corresponds to lateral displacement, varies less across these movement types and consequently provides a weaker regression target even under ideal data conditions.

7.4 Limitations of PCC as an Evaluation Metric

The trajectory comparison for subject S15 (Figure 4) illustrates a fundamental limitation of PCC as the sole evaluation metric for trajectory reconstruction. Despite achieving a mean PCC of 0.917 — the highest in the dataset — the predicted trajectory occupies a much larger spatial region than the actual trajectory, which is tightly localized. PCC measures linear correspondence between the time series of each coordinate independently, so a model can score highly by correctly tracking the direction of change while still producing spatially inaccurate predictions with excessive amplitude. A metric such as mean Euclidean distance between predicted and actual trajectory points would more faithfully capture the spatial fidelity of reconstruction and is recommended for future work.

7.5 Deviations from the Original Paper

Several implementation choices deviated from the method described in Li et al. (2024), which account for the gap between our mean PCC of 0.460 and the paper's reported 0.511 ± 0.019:

- **Validation scheme:** The paper uses 5-fold cross-validation across 10 runs (8 training, 2 test per fold), whereas our implementation uses a single 80/20 random train-test split. Cross-validation produces a more robust performance estimate and typically yields higher reported PCC due to averaging over multiple folds.
- **Regression method:** The paper applies standard polynomial regression (ordinary least squares). Our implementation substitutes Ridge Regression ($\alpha = 10.0$) to prevent numerical instability caused by the high-dimensional third-order polynomial feature space. Ridge regularization shrinks coefficients and can reduce overfitting but may also reduce peak correlation when data is limited.
- **Reduced training data:** Due to widespread `handPosX` corruption, most subjects had only 2 valid runs available compared to the paper's full 10 runs. This substantially reduces the number of training trials per class and limits the reliability of the fitted models.

- **Channel selection:** The paper applies a formal feature selection step in the temporal domain before PCA. Our implementation applies PCA directly on all 31 channels without a prior temporal feature selection stage, which may retain more redundant information in the feature space.

Despite these differences, the overall mean PCC of 0.460 falls within a broadly comparable range to the paper's result, validating the general feasibility of the TFP approach for EEG-based 3D trajectory reconstruction.

8 Limitations

8.1 Data Availability

The most significant practical limitation of this implementation was the widespread corruption of the `handPosX` channel across the dataset. With the majority of runs discarded for most subjects, the effective training set was reduced to as few as 2 runs per subject in many cases. This is far below the 8 training runs per subject used in the original paper, and directly limits the reliability of the fitted regression models. Subjects S03 and S13 could not be evaluated at all. Future work should investigate whether alternative interpolation or imputation strategies could recover partially corrupted runs rather than discarding them entirely.

8.2 Validation Scheme

The paper employs 5-fold cross-validation across the 10 runs, providing a statistically robust performance estimate averaged over multiple train-test splits. Our implementation uses a single 80/20 random train-test split, which is more sensitive to the particular trials that happen to fall in the test set and tends to underestimate true generalization performance. Adopting proper cross-validation in future implementations would yield more reliable and comparable PCC estimates.

8.3 Regression Approach

The original TFP method uses ordinary least squares polynomial regression. We substituted Ridge Regression ($\alpha = 10.0$) to prevent numerical instability arising from the high-dimensional cubic polynomial feature space. While this prevents coefficient explosion, the regularization also introduces a bias that can reduce peak correlation, particularly when training data is limited. An adaptive choice of the regularization parameter — for instance, selected via cross-validation — could improve performance.

8.4 Evaluation Metric

PCC measures linear correspondence between predicted and actual coordinate time series independently along each axis. As demonstrated by the trajectory comparison for subject S15, a model can achieve high PCC while producing spatially inaccurate 3D trajectories with incorrect amplitude. The absence of a spatial error metric such as mean Euclidean distance or dynamic time warping distance means that the quality of 3D trajectory reconstruction is not fully captured by the reported numbers alone.

8.5 Computational Constraints

The pipeline was executed on Google Colab, which imposes memory and runtime limitations. To manage memory, intermediate arrays were explicitly deleted and garbage collection was triggered between subjects. The third-order polynomial feature expansion is computationally expensive for large K (number of PCA components), and the per-trial model fitting approach scales linearly with the number of training trials. On a system with more memory, batching strategies or vectorized fitting could significantly reduce runtime.

8.6 Generalizability

The pipeline was evaluated on a single public dataset (ULM, BNCI Horizon 2020) under controlled laboratory conditions. Performance on data collected in more naturalistic settings, with different electrode montages, or from patient populations with motor impairments, may differ substantially. Additionally, the class-specific model architecture assumes that the movement class label is known at test time, which is not a realistic assumption in a closed-loop BCI system where the class must itself be inferred from EEG.

9 Conclusion

This term paper presented a full implementation of an EEG-based continuous 3D hand trajectory decoding pipeline, closely following the TFP method proposed by Li et al. (2024). The pipeline incorporated ICA-based artifact removal, time window fixation, PCA-based channel dimensionality reduction, and class-specific third-order polynomial regression with Ridge regularization, evaluated on the publicly available upper limb movement dataset from 15 subjects.

The implementation successfully decoded continuous 3D hand trajectories for 13 of the 15 subjects, achieving a mean Pearson Correlation Coefficient of 0.460 across all subjects and dimensions. The Y dimension was decoded most reliably (mean PCC

0.594), followed by Z (0.424) and X (0.362). The lower X-dimension performance was traced to widespread corruption of the `handPosX` kinematic channel in the dataset, which forced the discard of the majority of runs for most subjects and severely restricted the available training data. Despite this, the overall result is broadly comparable to the paper's reported mean PCC of 0.511 ± 0.019, validating the general feasibility of the approach.

Substantial inter-subject variability was observed, with per-subject mean PCC ranging from -0.260 (S07) to 0.917 (S15). Visual inspection of predicted trajectories revealed that PCC alone is an incomplete measure of reconstruction quality: even the best-performing subject showed large spatial discrepancies between the predicted and actual 3D path, motivating the use of additional spatial error metrics in future evaluations.

Key deviations from the original paper — including the use of a single train-test split instead of 5-fold cross-validation, Ridge Regression instead of ordinary least squares, and severely reduced training data due to missing kinematic labels — collectively account for the performance gap relative to the published results. Addressing these deviations in future work, alongside collecting a cleaner dataset with complete kinematic recordings, would be expected to bring performance closer to the reported benchmark.

Overall, this work demonstrates that nonlinear polynomial regression applied to PCA-reduced EEG features is a viable and computationally tractable approach to continuous limb trajectory decoding, with clear potential for application in motor rehabilitation BCIs for patients with upper limb impairments.

10 References

1. Li, S., Tian, M., Xu, R., Cichocki, A., & Jin, J. (2024). Decoding continuous motion trajectories of upper limb from EEG signals based on feature selection and nonlinear methods. *Journal of Neural Engineering*, **21**(6), 066039. `https://doi.org/10.1088/1741-2552/ad9cc1`

2. Ofner, P., Schwarz, A., Pereira, J., & Müller-Putz, G. R. (2017). Upper limb movements can be decoded from the time-domain of low-frequency EEG. *PLoS ONE*, **12**(8), e0182578.

3. Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C., ... & Hämäläinen, M. (2013). MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience*, **7**, 267.

4. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of*

Machine Learning Research, **12**, 2825–2830.

5. Iriarte, J., Urrestarazu, E., Valencia, M., Alegre, M., Malanda, A., Viteri, C., & Artieda, J. (2003). Independent component analysis as a tool to eliminate artifacts in EEG: a quantitative study. *Journal of Clinical Neurophysiology*, **20**(4), 249–257.

6. McDonald, G. C. (2009). Ridge regression. *Wiley Interdisciplinary Reviews: Computational Statistics*, **1**(1), 93–100.

7. Daly, J. J., & Wolpaw, J. R. (2008). Brain-computer interfaces in neurological rehabilitation. *The Lancet Neurology*, **7**(11), 1032–1043.

8. Bradberry, T. J., Gentili, R. J., & Contreras-Vidal, J. L. (2010). Reconstructing three-dimensional hand movements from noninvasive electroencephalographic signals. *Journal of Neuroscience*, **30**(9), 3432–3437.